

ALGORITHMIQUE ET PROGRAMMATION

Pr Amadou Dahirou GUEYE

Enseignant-chercheur en Informatique, UFR STA - UAM

Contact : dahirou.gueye@uam.edu.sn

Chapitre 2 : Les structures de contrôle

- ❑ Lors de la rédaction d'un algorithme, il faut être très explicite et indiquer clairement les conditions et l'ordre dans lesquels les actions envisagées doivent être effectuées.
- ❑ Les directives d'un algorithme qui sont utilisées à cette fin sont appelées structure de contrôle. Elles permettent de décrire la structure logique d'un algorithme et facilitent sa traduction en programme.
- ❑ Lorsqu'elles sont traduites dans un langage de programmation, les structures de contrôle prennent le nom d'instructions de contrôle.
- ❑ Les structures de contrôle de base sont l'alternative, la répétition, la structure de choix. Elles sont immédiatement disponibles sous forme d'instructions de contrôle dans les langages de programmation comme Pascal et python



Partie 1 : Les structures de contrôle en algorithme et en langage Pascal

e

Ecrivons l'algorithme permettant de calculer les valeurs de la fonction $f(x) = |x|$ pour un réel x donné.

Pour calculer $f(x)$, on doit tester le signe de x et effectuer l'une des deux actions suivantes :

$f(x) = x$ (si $x \geq 0$) et $f(x) = -x$ (si $x < 0$)

La structure de contrôle permettant de programmer ceci est l'alternative, dont la forme générale est :

si condition (booléenne) **alors**

action 1

Sinon

action 2

Cela nous donne pour notre exemple :

si $x \geq 0$ alors

$f(x) = x$

sinon

$f(x) = -x$

Remarque : Dans le cas où la 2^{ème} action est vide, on peut omettre le SINON.

En Turbo Pascal, cela se traduit par l'instruction:

If condition **Then**

Instruction(s)

else

Instruction(s) ;

Remarques :

- Il ne faut jamais faire précéder un *else* par un *point-virgule*.
- S'il y a plus d'une instruction à exécuter, il faut les encadrer par **begin** et **end** :

If condition **Then**

begin

Instructions

end

else

begin

Instructions

end;

Exercice d'application:

Une société paie ses employés chaque semaine. Le salaire étant calculé sur la base d'un taux horaire, écrire un programme qui calcule et affiche le salaire sachant qu'il y a une majoration de 20% si le nombre d'heures est supérieur à 35.

Program salaire ;

Var nbre_heures, taux_horaire : integer ;
 salaire_hebdo : real ;

begin

Write ('donnez le nombre d"heures:');

readln (nbre_heures) ;

write ('donnez le taux horaire') ; readln (taux_horaire) ;

if nbre_heures<= 35 then (* pas de majoration *)

 salaire_hebdo := nbre_heures*taux_horaire

else (* majoration de 20%, i.e. 0,2 du salaire hebdomadaire*)

 salaire_hebdo := nbre_heures*taux_horaire* 1.2) ;

writeln ('Le salaire hebdomadaire est de ', salaire_hebdo) ;

end.

2 La répétition

Elle permet d'effectuer plusieurs fois consécutives la même opération. Sa forme la plus simple est :

répéter

action

Mais un programme simulant une telle opération ne s'arrêterait, en principe jamais.

Donc il nous faut pouvoir contrôler une répétition. On dispose pour cela de deux moyens :

- ajouter à cette structure une condition permettant de décider s'il faut poursuivre ou arrêter l'exécution de l'action répétée,
- prescrire le nombre de fois qu'il s'agira d'exécuter l'action.

2.1 La boucle avec compteur

La forme générale est

pour <compteur> := <valeur initiale> **à** <valeur finale> **faire**

action (à répéter)

Un pas (généralement égal à 1) permet de faire passer automatiquement le compteur d'une valeur initiale à une valeur finale, et à chaque étape (ou itération ou tour de boucle), on exécute l'action spécifiée.

Ce pas, appelé aussi incrément, peut être négatif ; cela sous entend que la valeur initiale est supérieure à celle finale.

Il est à remarquer que cette structure de contrôle s'apparente beaucoup avec la formule mathématique Σ .

Exemple :

Voici un algorithme qui affiche les entiers compris entre 1 et 100.

variable i : entier

debut

 pour i :=1 à 100 faire

 ecrire(i)

fin

En Pascal, la boucle avec compteur se traduit
par l'instruction

for <compteur> **:=** <valeur initiale> **to** <valeur finale> **do**

 instruction(s)

S'il y a plus d'une instruction à exécuter, il faut les encadrer par **begin** et **end** :

for <compteur> **:=** <valeur initiale> **to** <valeur finale> **do**

begin

instructions

end ;

Lorsque la valeur initiale est supérieure à la valeur finale, le **to** doit être remplacé par **downto**.

Exemple :

Voici la traduction de l'algorithme précédent :

```
var i : integer;  
begin  
  for i :=1 to 100 do  
    writeln(i) ;  
  end.
```

Exercice d'application:

Ecrire un programme qui calcule la somme des n premiers entiers, n donné.

```
var i, n, s : integer;
```

```
begin
  readln(n);
  s := 0;
  for i := 1 to n do
    s := s + i;
  writeln(s);
end.
```

2.2 La boucle TANT QUE

Elle s'écrit sous la forme

tant que <condition> **faire**
 <instruction(s)>

et permet de contrôler la répétition et de garantir son achèvement.

Par exemple, pour afficher dix fois le message 'Bonjour à tous les nouveaux programmeurs !', on peut utiliser la suite d'instructions:

Nbfois := 0 ;

Tant que Nbfois < 10 **faire**

Debut

Ecrire ('Bonjour à tous les nouveaux programmeurs !')

Nbfois := Nbfois + 1

Fin

Remarques :

1. Dans une boucle **tant que**, la condition contrôlant la répétition est testée avant chaque itération i.e. l'exécution du corps de la boucle.
2. Lorsque le corps de la boucle contient plus d'une instruction (comme dans l'exemple ci-dessus), il faut l'encadrer par les termes *debut* et *fin*.

En Pascal, cela s'écrit :

while <condition> **do**

 <instruction>

ou

while <condition> **do**

begin

 <instructions>

end

Exercice d'application

Ecrire un programme permettant la saisie de la réponse à la question : « Aimez vous l'algorithmique (o/n) ? ». Le programme doit afficher un message en cas de mauvaise réponse, et renouveler la question jusqu'à ce que la réponse soit correcte.

```
Var rep : char ;
```

```
Begin
```

```
write('Aimez vous l'algorithmique (o/n) ? ');
```

```
readln(rep);
```

```
while (rep<>'o') and (rep<>'n') do
```

```
begin
```

```
    write('Veuillez répondre par oui(o) ou non(n) svp');
```

```
    write('Aimez vous l'algorithmique (o/n)? ');
```

```
    readln(rep);
```

```
end;
```

```
end.
```

2.3 La boucle REPETER

Elle s'écrit

Repeter

<instruction(s)>

jusqu'à <condition>;

Elle est presque identique à la boucle **tant que** à la différence près que la condition de sortie est testée après exécution des instructions.

Dans une boucle **repeter**, on est sûr que le corps de la boucle sera exécuté au moins une fois. Donc, on choisira cette structure de contrôle si l'on veut que le corps de la boucle soit exécuté au moins une fois. Ici, même si le corps de la boucle contient plus d'une instruction, il n'est pas nécessaire de l'encadrer par *debut* et *fin*.

Par exemple, si l'on devait rédiger l'algorithme de l'exercice précédent en utilisant l'instruction **repeter** :

Variable rep : caractère ;

debut

repeter

 ecrire('Aimez vous l'algorithmique (o/n) ? ');

 lire(rep) ;

jusqu'à (rep='o') ou (rep='n');

fin.

En Pascal, cette structure se traduit par l'instruction :

repeat

<instruction(s)>

until <condition>

Exercice d'application

Ecrire un programme qui permet de saisir une valeur entière positive et de calculer sa racine carrée.

Var entier_positif : integer ; Racine :real ;

Begin

repeat

write('donner un entier positif :) ; Readln(entier_positif) ;

Until entier_positif > 0;

Racine := sqrt(entier_positif) ;

{sqrt est une fonction prédéfinie en Pascal, elle calcule la racine carrée d'un entier positif}

writeln('la racine carrée de ',entier_positif, ' est ', racine) ;

End.

Remarque:

On utilise généralement les instructions **while** ou **repeat** lorsque l'on ne connaît pas, à l'avance, le nombre d'itérations

3 La structure de choix

Considérons le programme qui permet d'afficher littéralement un chiffre saisi au clavier.

On pourrait l'écrire en utilisant l'alternative,
on imbriquerait alors des *Si* :

Lire (X)

Si $X = 1$ alors écrire ('un') Sinon

Si $X = 2$ alors écrire ('deux') sinon

Si $X = 3$ alors écrire ('trois') sinon

Si $X = 4$ alors écrire ('quatre') sinon

Si $X = 5$ alors écrire ('cinq') sinon

Si $X = 6$ alors écrire ('six') sinon

Si $X = 7$ alors écrire ('sept')

sinon

Si $X = 8$ alors écrire ('huit') sinon

Si $X = 9$ alors écrire ('neuf') sinon

Si $X = 0$ alors écrire ('zero')

La structure de choix dont la forme générale est la suivante est plus adaptée à ce problème.

Suivant <expression> **faire**

Valeur 1 : action 1

Valeur 2 : action 2

...

Valeur n : action n

Sinon : action par défaut

Fin

Son effet est de réaliser l'action correspondant à une *condition i* qui est satisfaite ; si aucune des conditions n'est satisfaite, c'est la condition par défaut qui est réalisée.

condition i signifie ici : *expression = valeur i*

L'expression peut être un identificateur.

En Pascal cette la structure de choix est codée par l'instruction **case** :

case <expression> **of**

Valeur 1 : instruction(s) 1;

Valeur 2 : instruction(s) 2 ;

...

Valeur n : instruction(s) n ;

else instruction(s) par défaut

end

S'il y a plus d'une instruction à exécuter, il faut les encadrer par **begin** et **end**.

Maintenant, reprenons l'exemple précédent en Pascal:

Readln(x) ;

case x **of**

1 : writeln('un');

```
2 : writeln('deux');  
3 : writeln('trois');  
4 : writeln('quatre');  
5 : writeln('cinq');  
6 : writeln('six');  
7 : writeln('sept');  
8 : writeln('huit');  
9 : writeln('neuf');  
0 : writeln('zéro')  
  
else writeln('ce n'est pas un chiffre ');  
end;
```

4 Exercices proposés

Exercice 1

Un billet d'avion Dakar-Paris-Dakar de la compagnie Air Sanar coûte 375 000F CFA. Cependant, une réduction est accordée sur ce billet pour les voyageurs de moins de 25 ans (20% de réduction) et pour les voyageurs de plus de 65 ans (15% de réduction).

Ecrire un programme Pascal qui calcule le prix du billet pour un voyageur donné.

Exercice 2

- a) Que réalise la séquence algorithmique suivante ?

A := 2

B := 2

REPETER

A := A + 2

B := B + 1

Afficher A

Afficher B

JUSQU'A (A+B >= 10)

- b) Traduire cet algorithme en Pascal en complétant (déclarations des variables, etc.).
- c) Reprendre le programme précédent en utilisant une boucle WHILE.
- d) Quelles sont les différences entre ces deux instructions ?
Qu'est-ce qui les différencie de la boucle FOR ?

Exercice 3

- a) Ecrire un programme qui effectue la multiplication de deux entiers positifs saisis

au clavier, par additions successives. *Par exemple, $6 \times 3 = 3 + 3 + 3 + 3 + 3 + 3$*

b) Améliorer le programme du a) en

choisissant le plus petit élément des deux nombres pour les itérations et en introduisant un contrôle de saisie pour obliger la saisie de nombres strictement positifs.

c) Exercice 4

Ecrire un programme Pascal qui, à partir de la saisie du prix unitaire d'un produit (PU) et de la quantité commandée (QTCOM), affiche le prix à payer (PAP) en détaillant le port (PORT) et la remise (REM), sachant que:

-le port est gratuit si le prix des produits (TOT) est supérieur à 5 000 F. Dans le cas contraire, le port est de 2% de TOT.

-la remise est de 5% si TOT est compris entre 2 000 et 10 000 F et de 10% au-delà.

Exercice 5

Ecrire un algorithme qui permet de calculer et d'afficher la date du lendemain connaissant la date du jour (par exemple si la date du jour est 31/12/1998, la date du lendemain sera 01/01/1999).
Coder cet algorithme en Turbo Pascal.

Exercice 6

Ecrire un programme qui calcule la moyenne de notes lues au clavier. Le nombre de notes n'est pas connu à l'avance et l'utilisateur peut en fournir autant qu'il le désire sans toutefois dépasser 1000 notes. Pour signaler qu'il a terminé, on convient qu'il fournira une note fictive négative. Celle-ci ne devra naturellement pas être prise en compte dans le calcul de la moyenne.



Partie 2 : La structure alternative, les listes et les structures répétitives en langage Python:

Sélection conditionnelle

if condition :

commande exécutée si la condition est vraie

commande exécutée si la condition est vraie...

suite du programme (exécuté que la condition soit vraie ou fausse)

- Attention à l'indentation (= les espaces blancs en début de ligne)
- La condition est une comparaison utilisant un booléen ou un opérateur :

Inférieur à : <

Supérieur ou égal à : >=

Supérieur à : >

Égal à : ==

Inférieur ou égal à : <=

Différent de : !=

Sélection conditionnelle

- Le résultat doit être ceci:

```
>>> a=120
>>> if(a>100):
...     print("a dépassé la centaine")
... 
```

- Frappez encore une fois <Enter>. Le programme s'exécute, et vous obtenez :

```
a dépassé la centaine
>>> █
```


Instructions successives

- Il est possible d'ajouter plusieurs conditions successives, et un bloc par défaut (*else* = sinon)

if condition1:

commande exécutée si condition1 est vraie

commande exécutée si condition1 est vraie...

elif condition2:

commande exécutée si condition1 est fausse et condition2 est vraie...

else:

commande exécutée si condition1 et condition2 sont fausses...

suite du programme (exécutée que les conditions soient vraies ou fausses)

Instructions successives

■ Exemple1 de condition :

```
[>>> a=0
[>>> if a>0:
[...     print("a est positif")
[... elif a<0:
[...     print("a est négatif")
[... else:
[...     print("a est nul")
[... █
```

● Exemple2 de condition :

```
age = int(input("veuillez saisir votre âge : "))
```

```
if age == 0 :
```

```
    print("Vous êtes un nouveau-né.")
```

```
elif age < 18 :
```

```
    print("Vous êtes un enfant.")
```

```
elif age >= 65 :
```

```
    print("Vous êtes une personne âgée.")
```

```
else :
```

```
    print("Vous êtes un adulte.")
```

```
print("L'année prochaine vous aurez ", age + 1, " ans.")
```

Instructions imbriquées

- Il est possible d'imbriquer les unes dans les autres plusieurs instructions composées, de manière à réaliser des structures de décision complexes
- Exemple1:

```
[>>> if embranchement == "vertébrés":  
[...     if classe == "mammifères":  
[...         if ordre == "carnivores":  
[...             if famille == "félins":  
[...                 print("c'est peut-être un chat")  
[...             print("c'est en tous cas un mammifère")  
[...     elif classe == "oiseaux":  
[...         print("c'est peut-être un canari")  
... print("la classification des animaux est complexe")
```

Instructions imbriquées

■ Exemplez :

- ```
age = int(input("veuillez saisir votre âge : "))
poids = float(input("veuillez saisir votre poids : "))
```

```
if age == 0 :
 print("Vous êtes un nouveau-né.")
```

```
 if poids > 10.0 :
 print("Je pense qu'il y a une erreur sur le poids !")
```

## ■ Les opérateurs logiques **not**, **and** et **or** permettent de combiner plusieurs conditions entre elles.

◆ Respectivement NON, ET et OU logique

```
if (age < 18) or (age > 65) :
 print("Vous ne travaillez probablement pas.")
```

# Exercice 1 : les champignons

- On souhaite réaliser un programme Python permettant d'identifier des champignons. Afin de rester simple, nous nous limiterons aux 4 d'espèces de champignon suivant :
  - Les Ascomycètes (sans chapeau), comprenant :
    - Les morilles (avec pied)
    - Les truffes (sans pied)
  - Les Basidiomycètes (avec chapeau), comprenant :
    - Les bolets (sans lamelle)
    - Les autres champignons (avec lamelles)
- Le programme posera à l'utilisateur des questions de la forme
  - « Le champignon a-t-il un chapeau (o/n) ? », via la fonction `input()`

# Exercice 1 : les champignons

- Demander si le champignon a un chapeau :
  - S'il n'a pas de chapeau :
    - C'est un Ascomycète
    - Demander si le champignon a un pied
      - S'il a un pied, c'est une morille
      - S'il n'a pas de pied, c'est une truffe
  - S'il a un chapeau :
    - C'est un Basidiomycète
    - Demander si le champignon a des lamelles
      - S'il n'a pas de lamelles, c'est un bolet
      - S'il a des lamelles, c'est un autre champignon

# Listes

- Une liste est vue comme une collection d'éléments séparés par des virgules, l'ensemble étant enfermé dans des crochets

```
>>> jour = ['lundi', 'mardi', 'mercredi', 2020, 18.5, 'jeudi', 'vendredi']
>>> jour
['lundi', 'mardi', 'mercredi', 2020, 18.5, 'jeudi', 'vendredi']
>>> print (jour)
['lundi', 'mardi', 'mercredi', 2020, 18.5, 'jeudi', 'vendredi']
>>> █
```

- il est possible de changer les éléments individuels d'une

```
>>> print (jour)
['lundi', 'mardi', 'mercredi', 2020, 18.5, 'jeudi', 'vendredi']
>>> jour[3] = jour[3] + 37
>>> jour
['lundi', 'mardi', 'mercredi', 2057, 18.5, 'jeudi', 'vendredi']
>>> █
```

- Fonction len()

```
>>> print(len(jour))
```

# Listes

## ■ Fonction del

```
[>>> del(jour[4])
[>>> jour
['lundi', 'mardi', 'mercredi', 2057, 'jeudi', 'vendredi']
>>> █
```

## ■ Fonction append()

```
[>>> jour.append('samedi')
[>>> jour
['lundi', 'mardi', 'mercredi', 2057, 'jeudi', 'vendredi', 'samedi']
>>> █
```



# Accès aux éléments

## ■ Soit les listes

- >>> nombres = [15, 48, 20, 35]  
>>> animaux = ["éléphant", "girafe", "rhinocéros", "gazelle"]
- >>> liste3 = [2000, "Bambey", 3.14, ["Ablaye", "Daouda", 1980]]

| >>><br>print(nombres[2]) | >>><br>print(nombres[1:3]) | >>><br>print(nombres[2:3]) | >>><br>print(nombres[2:]) | >>><br>print(nombres[:2]) | >>><br>print(nombres[-1]) | >>><br>print(nombres[-2]) |
|--------------------------|----------------------------|----------------------------|---------------------------|---------------------------|---------------------------|---------------------------|
| 20                       | [48, 20]                   | [20]                       | [20, 35]                  | [15, 48]                  | 35                        | 20                        |

## ■ Notation

- la notation liste[i:j] désigne les éléments i à j-1 de la liste
- la notation [i:] désigne les éléments i et suivants
- la notation [:i] désigne les éléments 0 à i-1

# Les listes sont modifiables

## ■ Modification par affectation directe

### ○ Exemple1:

- >>> nombres[0] = 12
- >>> nombres
- [12, 48, 20, 35]

### ○ Exemple2:

- >>> liste3[3][1] = "Dahirou"  
>>> liste3
- [2000, "Bambey", 3.14, ["Ablaye", "Dahirou", 1980]]

# Les listes sont modifiables

■ Les listes peuvent être modifiées

- Exemples

```
animaux = ["éléphant", "girafe",
 "rhinocéros", "gazelle"]
```

- Ajouter un élément à la fin

```
animaux.append("lion")
```

- Ajouter un élément à une position donnée

```
animaux.insert(0, "hippopotame")
```

- Enlever un élément

```
animaux.remove("gazelle")
```

- Enlever un élément à une position donnée

```
del animaux[-2]
```

Que contient alors la liste animaux ?

# Les listes sont modifiables

- Ajouter tous les éléments d'une autre liste
- Trier une liste (ordre numérique / alphabétique)
- Concaténer deux listes
- Inverser l'ordre d'une liste
- Cloner une liste

## Exemples

```
animaux.extend(["lion",
"buffle"])
```

```
animaux.sort()
```

```
animaux + ["lion", "buffle"]
```

```
animaux.reverse()
```

```
Lcopie=list(animaux)
```

# Listes (Exemples (1))

- Supprimer une entrée avec son index avec la fonction **del**

```
>>> liste = ["a", "b", "c"]
```

```
>>> del liste[1]
```

```
>>> liste
```

```
['a', 'c']
```

- Supprimer une entrée avec sa valeur avec la méthode **remove**

```
>>> liste = ["a", "b", "c"]
```

```
>>> liste.remove("a")
```

```
>>> liste
```

```
['b', 'c']
```

# Listes (Exemples (2))

❑ Inverser les valeurs d'une liste avec la méthode **reverse**

```
>>> liste = ["a", "b", "c"]
```

```
>>> liste.reverse()
```

```
>>> liste
```

```
['c', 'b', 'a']
```

❑ Compter le nombre d'items d'une liste

```
>>> liste = [1,2,3,5,10]
```

```
>>> len(liste)
```

```
5
```

# Listes (Exemples (3))

❑ Compter le nombre d'occurrences d'une valeur

```
>>> liste = ["a","a","a","b","c","c"]
```

```
>>> liste.count("a")
```

```
3
```

```
>>> liste.count("c")
```

```
2
```

❑ Trouver l'index d'une valeur

```
>>> liste = ["a","a","a","b","c","c"]
```

```
>>> liste.index("b")
```

```
3
```

# Listes (Exemples (4))

❑ Trouver un item dans une liste avec le mot clé **in**

```
>>> liste = [1,2,3,5,10]
```

```
>>> 3 in liste
```

```
True
```

```
>>> 11 in liste
```

```
False
```

❑ La fonction range

```
>>> range(10)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```



# Listes (Exemples (5))

## ❑ Agrandir une liste

```
>>> x = [1, 2, 3, 4]
```

```
>>> y = [4, 5, 1, 0]
```

```
>>> x.extend(y)
```

```
>>> print x
```

```
[1, 2, 3, 4, 4, 5, 1, 0]
```

## ❑ Additionner deux listes

```
>>> x = [1, 2, 3]
```

```
>>> y = [4, 5, 6]
```

```
>>> x + y
```

```
[1, 2, 3, 4, 5, 6]
```

# Listes (Exemples (6))

❑ Multiplier une liste:

```
>>> x = [1, 2]
```

```
>>> x*5
```

```
[1, 2, 1, 2, 1, 2, 1, 2, 1, 2]
```

❑ Initialiser une liste:

- ```
>>> [0] * 5
```

- ```
[0, 0, 0, 0, 0]
```

# Listes (Exemples (7))

❑ Obtenir le plus grand'élément d'une liste, ou le plus petit :

- `>>> max([2, 1, 4, 3])`

- 4

- `>>> min([2, 1, 4, 3])`

- 1

❑ Faire la somme d'une liste:

- `>>> sum([2, 1, 4, 3])`

- 10

# Listes et chaînes de caractères

- Couper une chaîne de caractères en une liste de chaînes :

`split`

- ◆ `split` sans argument : couper sur les espaces, sauts de ligne, tabulations

```
>>> "Une phrase avec des mots".split()
```

```
["Une", "phrase", "avec", "des", "mots"]
```

- ◆ Split avec argument : couper sur un séparateur donné

```
>>> "Plan-Jaxaay".split("-")
```

```
["Plan", "Jaxaay"]
```

- Assembler une liste de chaînes en une seule chaîne : `join`

```
>>> "/" .join(["répertoire", "sous_répertoire", "fichier"])
```

```
"répertoire/sous_répertoire/fichier"
```

# Boucles

- Une boucle permet d'exécuter plusieurs fois les mêmes commandes

- La boucle parcourt une séquence

**for variable in sequence :**

*commande répétée plusieurs fois*

*commande répétée plusieurs fois*

*suite du programme (exécuté une seule fois)*

# Boucles

- La séquence peut être :
  - une variable
    - Chaîne de caractères
    - Liste, ensemble, tuple
    - ...
  - une liste d'indice générée avec la fonction range()  
`range(5)`    => [0, 1, 2, 3, 4]

# Boucles

- Exemple de séquence de type chaîne de caractères:

```
[>>> prenom="Dahirou"
[>>> for car in prenom:
[... print(car+' *',end=' ')
[...
D * a * h * i * r * o * u * >>>
```

- L'instruction **for** permet donc d'écrire des boucles, dans lesquelles l'itération traite successivement tous les éléments d'une séquence donnée.
- L'argument **end = " "** signifie que vous souhaitez remplacer le saut à la ligne par un simple espace. Si vous supprimez cet argument, les nombres seront affichés les uns en-dessous des autres (sur la verticale).

# Boucles

## ■ Exemple de séquence de type Liste

```
[>>> liste=['SATIC','SDD','ECOMIJ']
[>>> for ufr in liste:
[... print('longueur de la chaine',ufr,'=',len(ufr))
[...
longueur de la chaine SATIC = 5
longueur de la chaine SDD = 3
longueur de la chaine ECOMIJ = 6
>>> █
```

**environnements = ["savane", "forêt", "savané", "rivière"]**

◆ Afficher un animal par ligne

```
for animal in animaux : print(animal)
print("C'est fini !")
```

◆ Afficher un animal par ligne avec son numéro

```
for i in range(len(animaux)) :
 print("Numéro " + str(i) + " : " + animaux[i])
print("C'est fini !")
```

◆ Afficher un animal par ligne avec son environnement

```
for i in range(len(animaux)) :
 print(animaux[i] + " dans la " + environnements[i])
```



# Boucles (boucler sur plusieurs listes avec zip)

## ■ Exemples

**animaux = ["éléphant", "biche", "rhinocéros", "brochet"]**

**environnements = ["savane", "forêt", "savane", "rivière"]**

- for animal, enviro in zip(animaux, environnements):
- print(animal + " vit dans " + enviro)

## ■ Ce qui donne :

- éléphant vit dans savane
- biche vit dans forêt
- Rhinocéros vit dans savane
- brochet vit dans rivière

# Boucles (boucler sur plusieurs listes avec des boucles imbriquées)

## ■ Exemples

**animaux = ["éléphant", "biche", "rhinocéros", "brochet"]**

**environnements = ["savane", "forêt", "savane", "rivière"]**

- for animal in animaux:
- for env in environnements:
- print (animal+ " dans " + env )

## ■ Ce qui donne :

- éléphant dans savane
- éléphant dans forêt
- éléphant dans rivière
- biche dans savane
- biche dans forêt
- biche dans rivière

rhinocéros dans savane  
rhinocéros dans forêt  
rhinocéros dans rivière  
brochet dans savane  
brochet dans forêt  
brochet dans rivière

# Boucles

- **break** permet d'interrompre la boucle
- **continue** permet de passer immédiatement à l'élément suivant
- La boucle **for** peut aussi avoir un bloc **else** :
  - for variable in liste :**
    - if condition d'exclusion :**
      - continue**
      - commande répétée plusieurs fois*
    - if condition :**
      - commande exécutée si condition satisfaite*
      - break**
  - else :**
    - commande exécutée si condition jamais satisfaite*

# Boucles

- **continue** : permet de passer immédiatement à l'élément suivant

**for** *variable* **in** *liste* :

**if** *condition d'exclusion* : **continue**

*commande répétée plusieurs fois*

```
for i in range(10):
 if i == 2:
 continue
 print(i)
```

# Boucles



**break** et **else** :



**break** permet d'interrompre la boucle

```
for i in range(10):
 if i == 2:
 break

 print(i)
```



le bloc **else** est exécuté si la boucle est allée jusqu'au bout (sans rencontrer **break**)

**for** *variable* **in** *liste* :

*commande répétée plusieurs fois*

**if** *condition* :

*commande exécutée si condition satisfaite*

**break**

**else** :

*commande exécutée si condition jamais satisfaite*

# Boucles

- La boucle **while** dure tant qu'une condition est satisfaite :  
**while condition :**

*commande répétée plusieurs fois*

*suite du programme (la boucle est finie)*

- Elle peut aussi avoir un **break** et un **else**
- Exemple: la liste des carrés et des cubes des nombres de 1 à 12

```
[>>> a=0
[>>> while a<12:
[... a=a+1
[... print(a , a**2 , a**3)
[...
```

- Afficher les dix premiers termes d'une suite appelée « Suite de Fibonacci »

# Exercices

- Exo1 : Afficher les dix premiers termes d'une suite appelée « Suite de Fibonacci ». Il s'agit d'une suite de nombres dont chaque terme est égal à la somme des deux termes qui le précèdent

- Résultat

```
...
1 2 3 5 8 13 21 34 55 89 >>> █
```

# Exercices

- Exo 2: Ecrivez un programme qui affiche les 20 premiers termes de la table de multiplication par 7.
- Exo 3: Ecrivez un programme qui affiche la suite de symboles suivante

- \*
- \*\*
- \*\*\*
- \*\*\*\*
- \*\*\*\*\*
- \*\*\*\*
- \*\*\*\*\*
- \*\*\*\*\*



# Exercices

- Exo 4: le dot-plot

- Le dot-plot est une technique très simple pour analyser visuellement deux séquences d'ADN

- Principe :

- ◆ Mettre une séquence à l'horizontal et l'autre en vertical, sur une matrice
- ◆ Mettre un X dans les cases où les bases sont les mêmes dans les deux séquences

- Faire un programme Python pour générer

Un dot-plot

- ◆ Les séquences d'ADN sont des chaînes de caractères
- ◆ On générera le dot-plot ligne par ligne et on écrira chaque ligne avec print()

|   | A | T | G | G | T | C | A | A | T |
|---|---|---|---|---|---|---|---|---|---|
| A | X |   |   |   |   |   | X | X |   |
| T |   | X |   |   | X |   |   |   | X |
| G |   |   | X | X |   |   |   |   |   |
| G |   |   | X | X |   |   |   |   |   |
| T |   | X |   |   | X |   |   |   | X |
| C |   |   |   |   |   | X |   |   |   |
| A | X |   |   |   |   |   | X | X |   |
| A | X |   |   |   |   |   | X | X |   |
| T |   | X |   |   | X |   |   |   | X |